

PRACTICAL - 01

```
#include <iostream>
```

```
#include <vector>
```

```
#include <cstdlib>
```

```
#include <ctime>
```

```
#include <chrono>
```

```
using namespace std;
```

```
using namespace std::chrono;
```

```
// Bubble Sort
```

```
void bubbleSort(vector<int> &arr)
```

```
{
```

```
    int n = arr.size();
```

```
    for(int i=0;i<n-1;i++)
```

```
        for(int j=0;j<n-i-1;j++)
```

```
            if(arr[j]>arr[j+1])
```

```
                swap(arr[j],arr[j+1]);
```

```
}
```

```
// Selection Sort
```

```
void selectionSort(vector<int> &arr)
```

```
{
```

```
    int n=arr.size();
```

```
    for(int i=0;i<n-1;i++)
```

```
    {
```

```
        int min=i;
```

PRACTICAL - 01

```
for(int j=i+1;j<n;j++)
    if(arr[j]<arr[min])
        min=j;

    swap(arr[i],arr[min]);
}
}
```

// Insertion Sort

```
void insertionSort(vector<int> &arr)
{
    int n=arr.size();

    for(int i=1;i<n;i++)
    {
        int key=arr[i];
        int j=i-1;

        while(j>=0 && arr[j]>key)
        {
            arr[j+1]=arr[j];
            j--;
        }

        arr[j+1]=key;
    }
}
```

PRACTICAL - 01

```
// Merge Sort

void merge(vector<int> &arr,int l,int m,int r)
{
    int n1=m-l+1;
    int n2=r-m;

    vector<int> L(n1),R(n2);

    for(int i=0;i<n1;i++)
        L[i]=arr[l+i];

    for(int j=0;j<n2;j++)
        R[j]=arr[m+1+j];

    int i=0,j=0,k=l;

    while(i<n1 && j<n2)
    {
        if(L[i]<=R[j])
            arr[k++]=L[i++];
        else
            arr[k++]=R[j++];
    }

    while(i<n1)
        arr[k++]=L[i++];

    while(j<n2)
```

PRACTICAL - 01

```
        arr[k++]=R[j++];
    }

void mergeSort(vector<int> &arr,int l,int r)
{
    if(l<r)
    {
        int m=(l+r)/2;

        mergeSort(arr,l,m);
        mergeSort(arr,m+1,r);

        merge(arr,l,m,r);
    }
}

// Quick Sort
int partition(vector<int> &arr,int low,int high)
{
    int pivot=arr[high];
    int i=low-1;

    for(int j=low;j<high;j++)
    {
        if(arr[j]<pivot)
        {
            i++;
            swap(arr[i],arr[j]);
        }
    }
}
```

PRACTICAL - 01

```
    }  
}  
  
swap(arr[i+1],arr[high]);  
  
return i+1;  
}  
  
void quickSort(vector<int> &arr,int low,int high)  
{  
    if(low<high)  
    {  
        int pi=partition(arr,low,high);  
  
        quickSort(arr,low,pi-1);  
        quickSort(arr,pi+1,high);  
    }  
}  
  
int main()  
{  
    const int n=100;  
  
    vector<int> arr(n),temp;  
  
    srand(time(0));  
  
    for(int i=0;i<n;i++)
```

PRACTICAL - 01

```
arr[i]=rand()%1000;
```

```
cout<<"Number of Elements = "<<n<<"\n\n";
```

```
auto start=high_resolution_clock::now();
```

```
temp=arr;
```

```
bubbleSort(temp);
```

```
auto stop=high_resolution_clock::now();
```

```
cout<<"Bubble Sort Time   : "<<duration_cast<microseconds>(stop-start).count()<<"  
microseconds\n";
```

```
start=high_resolution_clock::now();
```

```
temp=arr;
```

```
selectionSort(temp);
```

```
stop=high_resolution_clock::now();
```

```
cout<<"Selection Sort Time : "<<duration_cast<microseconds>(stop-start).count()<<"  
microseconds\n";
```

```
start=high_resolution_clock::now();
```

```
temp=arr;
```

```
insertionSort(temp);
```

```
stop=high_resolution_clock::now();
```

```
cout<<"Insertion Sort Time : "<<duration_cast<microseconds>(stop-start).count()<<"  
microseconds\n";
```

```
start=high_resolution_clock::now();
```

```
temp=arr;
```

```
mergeSort(temp,0,n-1);
```

```
stop=high_resolution_clock::now();
```

PRACTICAL - 01

```
    cout<<"Merge Sort Time    : "<<duration_cast<microseconds>(stop-start).count()<<"  
microseconds\n";
```

```
    start=high_resolution_clock::now();
```

```
    temp=arr;
```

```
    quickSort(temp,0,n-1);
```

```
    stop=high_resolution_clock::now();
```

```
    cout<<"Quick Sort Time    : "<<duration_cast<microseconds>(stop-start).count()<<"  
microseconds\n";
```

```
    return 0;
```

```
}
```